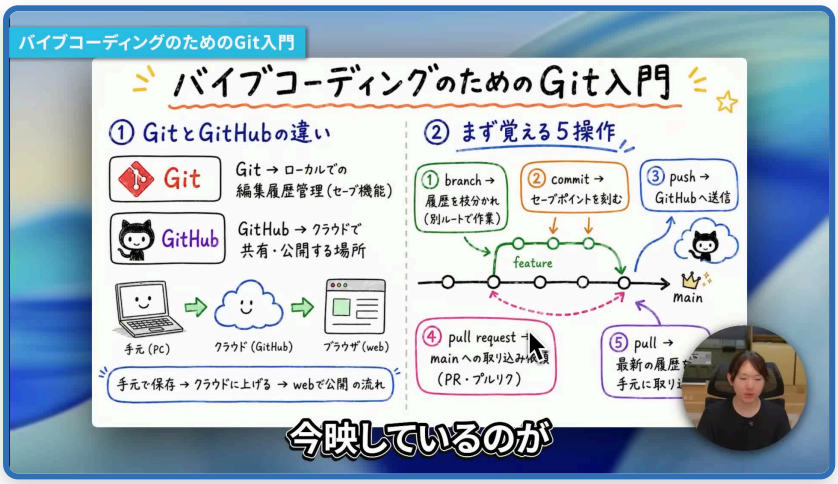
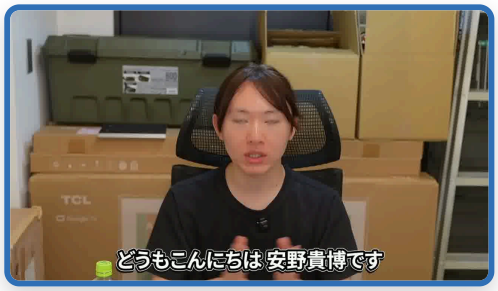
VIDEO DIGEST ／ 動画要約

バイブコーディングのための Git入門

安野貴博「バイブコーディング超入門 第5回」Git&GitHub解説編 — 1枚の説明図で学ぶ考え方と、Claude Codeでの短い実演



01:38　講義の全体を1枚にまとめた説明図。左が「GitとGitHubの違い」、右が「まず覚える5操作」



00:30　話者の安野貴博氏（字幕の表記）[00:30–00:57]

ひとことで言うと

Gitは手元のセーブ機能、GitHubはクラウドの共有・公開の場。操作はまず覚える5つに絞り、あとはAIの支援で使いこなせるのでは、というのが話者の見立てです。 [01:50–02:45] [03:06–04:03]

01

難関だが、越えると幅が広がる

壊れた機能を戻す、チームで開発する、世界に公開する。その中核にGit/GitHubがある。 [00:57–01:36]

02

5操作で一連の流れ

branch・commit・push・pull request・pull。枝分かれから最新の取り込みまで。 [09:50–10:06]

03

conflictは人間が決める

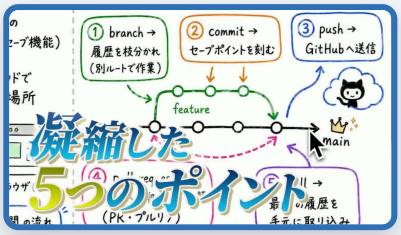
同じ項目を別々に変えると衝突し、どちらを採るか人が決めてマージする。 [10:06–11:31]

01 PART 1　00:00 – 01:36
なぜGitとGitHubを学ぶのか

オープニング
冒頭はダイジェスト。「Git&GitHub解説編」「凝縮した5つのポイント」「初学者を一気に進歩させるバイブル」とテロップで内容を予告します。
[00:00–00:30]



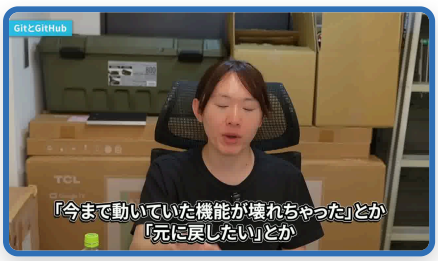
00:04　解説編のタイトル



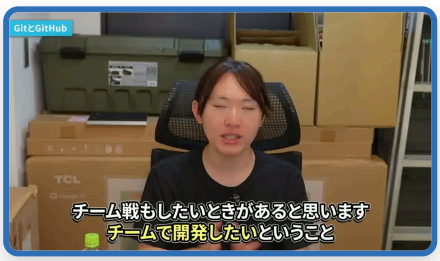
00:17　「凝縮した5つのポイント」

シリーズ第5回のテーマはGitとGitHub。話者は「かなりの難関」で難しいかもしれないと認めたうえで、ここを越えるとバイブコーディングでやっていることの幅を広げられる、と勧めます。 [00:30–00:57]

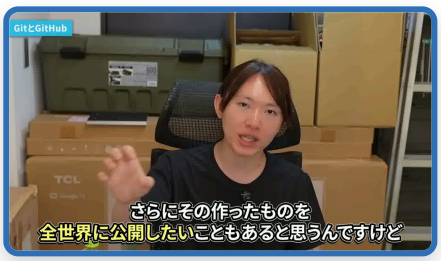
Git/GitHubが必要になる3つの場面



① 01:08　動いていた機能が壊れた。元に戻したい



② 01:17　作った機能を共有し、チームで開発したい



③ 01:22　作ったものを全世界に公開したい

どの場面も、GitとGitHubをしっかり使えるとできるようになる、と話者は説明しています。 [00:57–01:36]

> **いろんなことのコアになっていて、これができるだけで、できることがかなり広がる技術。**
>
> — 安野貴博氏（要旨）

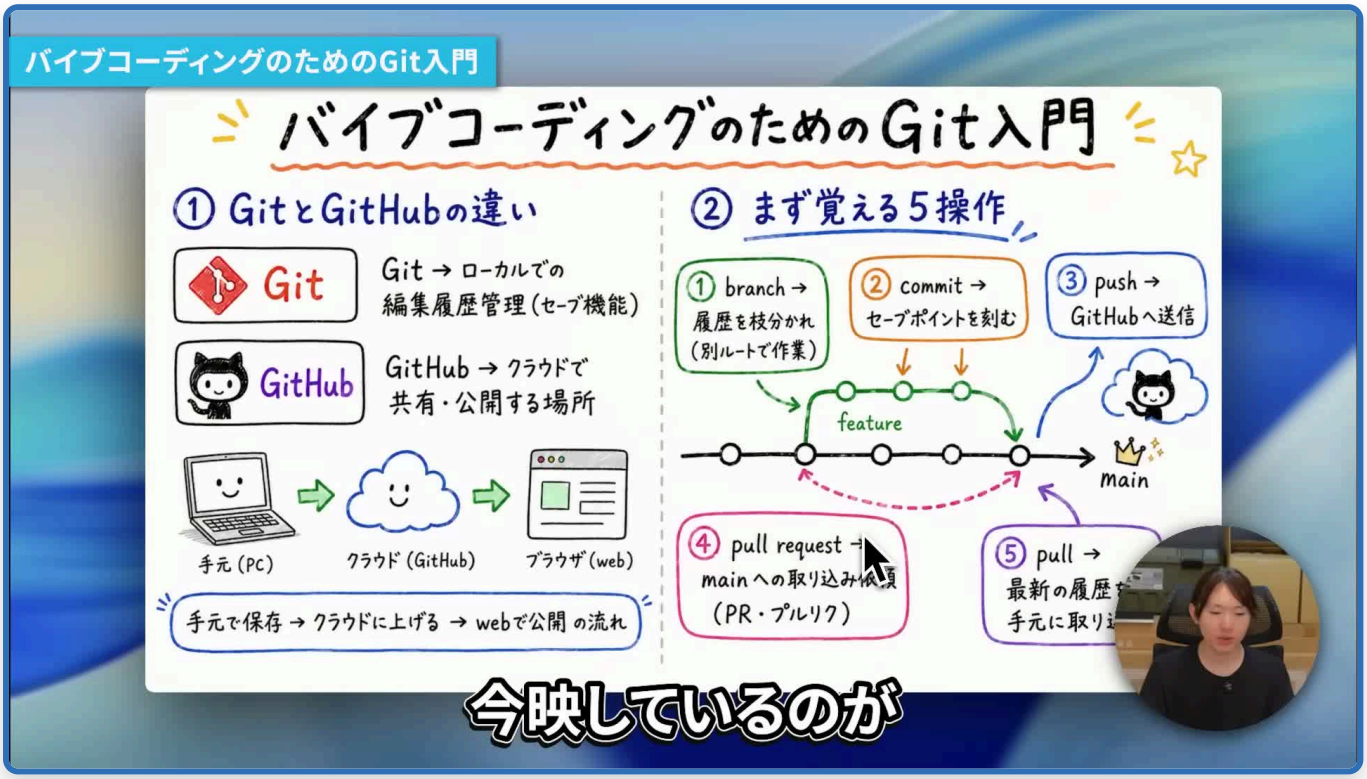


01:35　テロップ「出来ることが広がる技術」

[00:57–01:36]

02 PART 2　01:36 – 03:06
1枚の説明図が、講義の地図になる

この図は、話したい内容をChatGPTに入れて作ってもらった画像だと話者は説明しています。以降の解説はこの図の上で、黄色のハイライトや赤枠、マウスポインタで箇所を示しながら進みます。 [01:36–01:50]



01:38 「バイブコーディングのためのGit入門」。左に①GitとGitHubの違い、右に②まず覚える5操作

図の読み方

左半分：① GitとGitHubの違い
2つの役割と、手元(PC)→クラウド(GitHub)→ブラウザ(web)の流れ。 [01:36–01:50]

右半分：② まず覚える5操作
branch・commit・push・pull request・pullの5つの枠と、歴史の線。 [01:36–01:50]

横の線と小さな丸
左から右へ時間が進み、小さな丸がセーブポイント。丸を作るのがcommitです。 [04:03–06:13] [06:13–06:50]

黒い線「main」と緑の枝「feature」
mainはメインの歴史。featureは枝分かれした別ルートの作業です。 [07:11–08:03] [04:03–06:13]

PART 2 ／ ① GitとGitHubの違い

Git＝手元のセーブ、GitHub＝共有・公開の場

Git

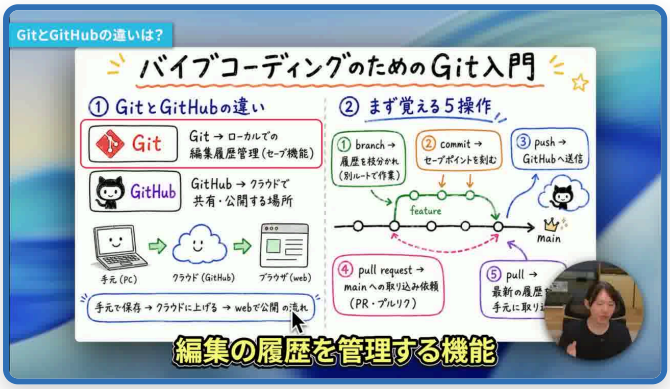
ローカルでの編集履歴管理（セーブ機能）

ローカルとは自分のパソコンのこと。バージョン管理、つまり今の状態を保存しておくセーブポイントの機能です。

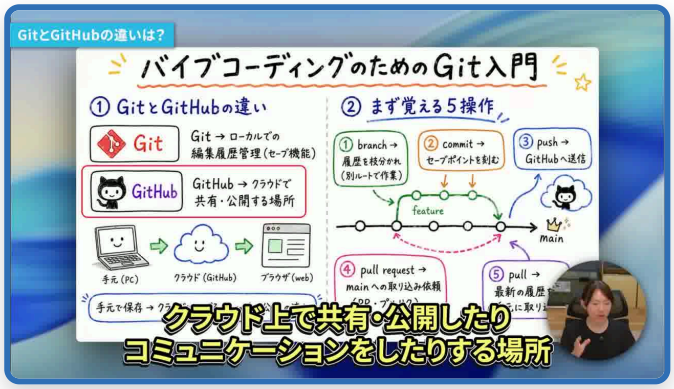
GitHub

クラウドで共有・公開する場所

Gitで作っているものを、クラウド上で共有・公開し、コミュニケーションする場所です。
[01:50–02:45]



02:05　Gitの行に赤枠。「編集の履歴を管理する機能」



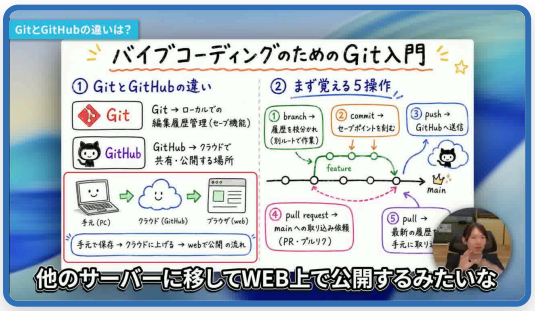
02:21　GitHubの行に赤枠。「クラウド上で共有・公開したり…」

POINT

Gitだけでもバージョン管理はできますが、業界標準としてエンジニアはGitHubにセーブポイントを上げて共有している、と話者は説明します。 [01:50–02:45]

両方を使うと「手元→クラウド→web」がつながる

手元（PC）	クラウド（GitHub）	ブラウザ（web）
パソコンで操作して保存	保存したものを上げる	他のサーバーに移して公開



02:58　流れの行に赤枠

図には「手元で保存→クラウドに上げる→webで公開の流れ」とあります。クラウド上のソースコードを他のサーバーに移してWebで公開するまでの一連の流れが、両方を使うとできるようになる、と話者は説明しています。 [02:45–03:06]

03 PART 3　03:06 – 10:06
削ぎ落として残した「まず覚える5操作」

話者によると、操作は本当は20個くらいある。超入門講座として削れるところを全部削り、それでも覚えておいてほしいものとして残したのが次の5つです。 [03:06–04:03]

約20	5
本来ある操作の数（話者の概算）	超入門として残した操作

① **branch**
履歴を枝分かれ

② **commit**
セーブポイントを刻む

③ **push**
GitHubへ送信

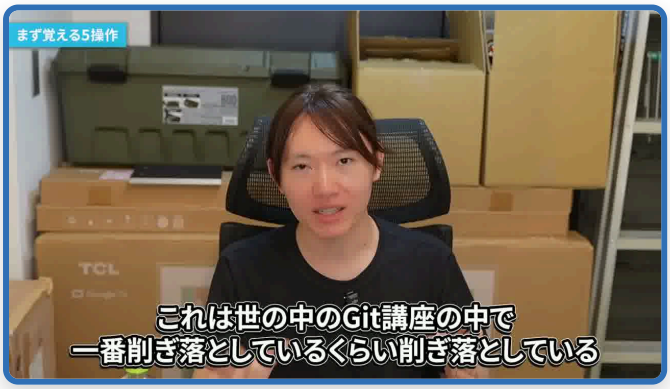
④ **pull request**
mainへの取り込み依頼

⑤ **pull**
最新の履歴を手元に取り込み

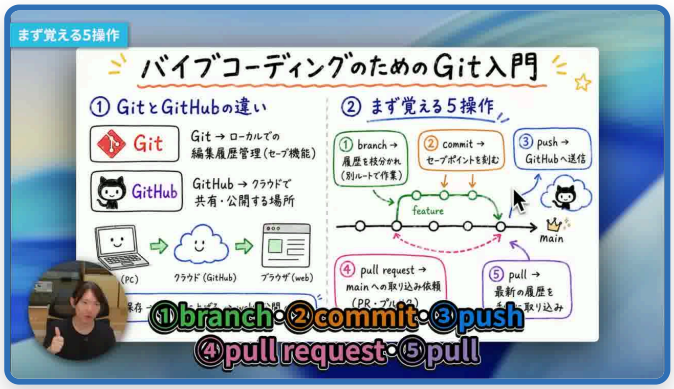
各操作の短い説明は、説明図の表記に基づきます。

> **ここまで削ぎ落としても、Claude CodeやCodexのようなAIの支援があったら、Gitを使いこなせるんじゃないか。**
>
> — 安野貴博氏（要旨）

[03:06–04:03]



03:28　「世の中のGit講座の中で一番削ぎ落としているくらい」



03:53　5つの操作名がテロップで並ぶ

① branch

履歴を枝分かれさせ、別ルートで作業する

QUESTION

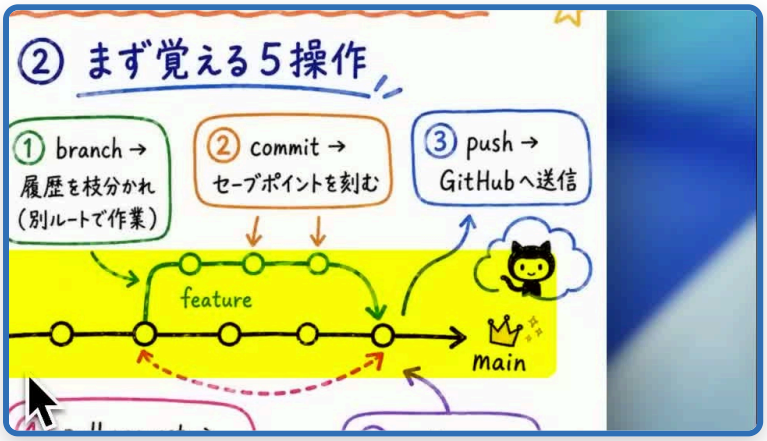
ゲームでいうと？

→

ボス戦の手前でセーブしておくこと

負けたらセーブポイントに戻って再挑戦する。同じように、新しい機能を実装する前に今の状態を保存し、いつでも戻れるようにします。

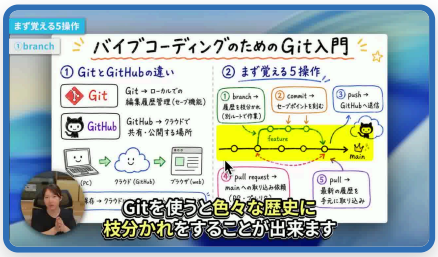
[04:03–06:13]



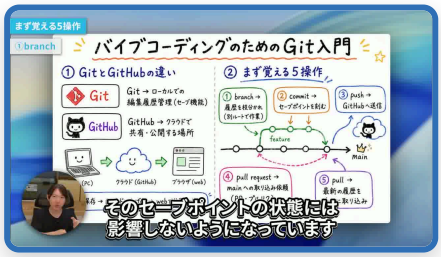
04:53 拡大画面。枝分かれしたfeatureからmainに戻るまでの範囲が黄色く強調される

- **時間は左から右へ。**小さな丸はセーブポイントのようなもの。
- **枝分かれ先の作業は、元のセーブポイントに影響しない。**ゲームで右に行くか左に行くかの分岐のように、ルートごとに分けられる。
- **うまくいけば元の歴史に戻せる。**いい機能ができたら、その歴史を正しい歴史として採用する。

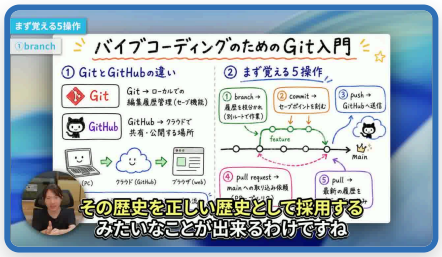
[04:03–06:13]



05:10 「色々な歴史に枝分かれをすることが出来ます」



05:45 「そのセーブポイントの状態には影響しない」



06:07 「その歴史を正しい歴史として採用する」

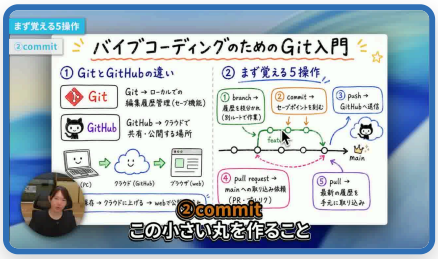
② commit

セーブポイントを刻む

commitは、図の小さな丸を作ること。今の状態をセーブポイントとしてどんどん刻んでいきます。ブランチを作ることは「branchを切る」と言います。 [06:13–06:50]

branchを切る → 編集 → commit（いい感じになったら） → また編集 → commit（こまめに繰り返す）

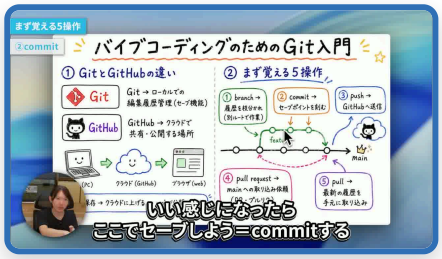
こまめにcommitしてセーブポイントを作りながら前に進むのが、Gitでの作業だと話者は説明しています。 [06:13–06:50]



06:16 「この小さい丸を作ること」



06:30 枝分かれを「branchを切る」と言う



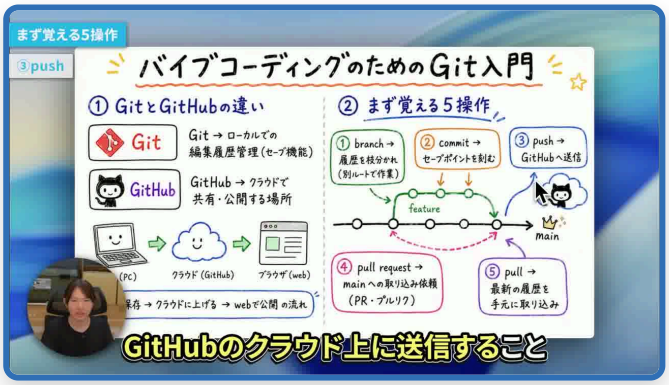
06:37 「いい感じになったら ここでセーブしよう＝commitする」

③ push

手元の作業をGitHubへ送信

push ＝「押し込む」

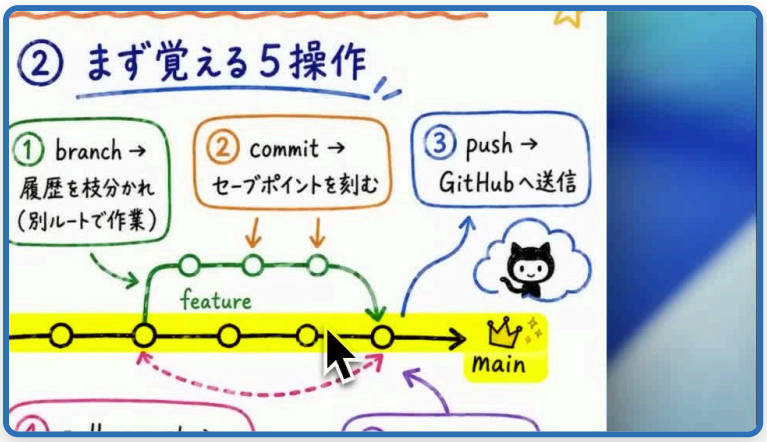
手元のパソコン上で作業しているものを、GitHubのクラウド上に送信します。これで手元でやった作業をGitHubにアップロードできます。 [06:50–07:11]



07:02 「GitHubのクラウド上に送信すること」

④ pull request

mainへの取り込み依頼（PR・プルリク）



07:26　拡大画面。黒い線（main）が黄色く強調される

黒い線＝main

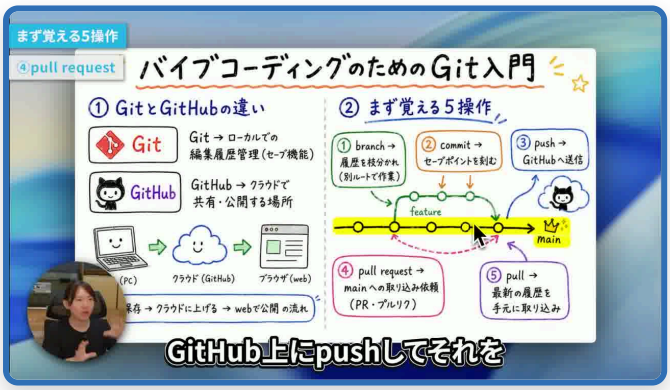
図で「main」と書かれた黒い線は、mainブランチ。その名の通りメインの歴史です。

[07:11–08:03]

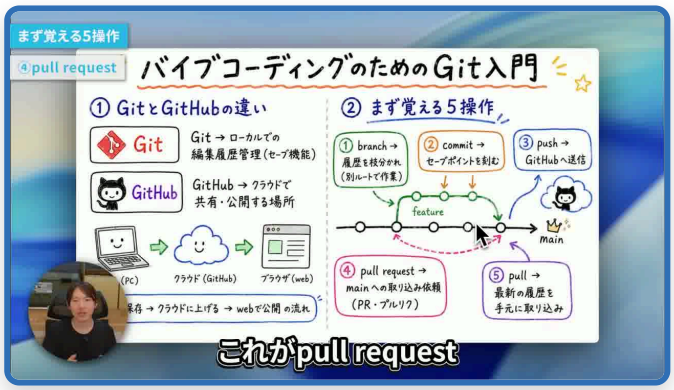
pull requestまでの流れ

自分の分岐で作業	GitHubにpush	pull request
機能を作る		mainに入れてと依頼

自分の手元で枝分かれして作った機能をGitHubにpushし、それをmainの歴史に入れてほしいとリクエストするのがpull requestです。 [07:11–08:03]



07:36　「GitHub上にpushしてそれを」



07:52　「これがpull request」

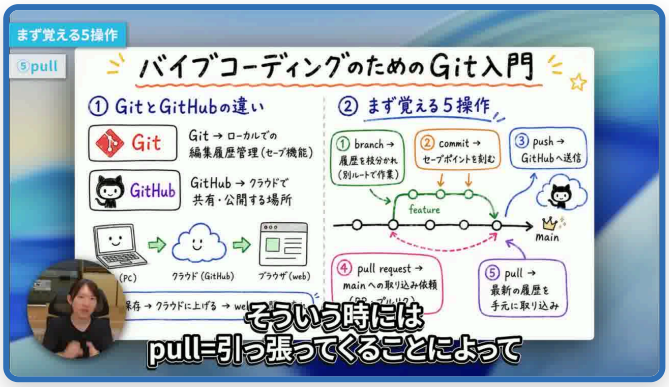
> **POINT**
>
> **pull requestは「プルリク」「PR」とも呼ばれ、GitHub上で中身を見たり、レビューしたりできます。** [07:11–08:03]

⑤ pull

最新の履歴を手元に取り込み

①〜④で、新しく分岐を作ってセーブし、クラウドにアップロードして、mainに入れ込むところまで進みました。そのクラウド上のmainに入った新しい歴史を、自分のパソコンへ「引っ張ってくる」のがpullです。これで手元に最新のmainの状態を持ってこられます。 [08:03–08:45]

音声の「10から40まで」は、字幕の「①〜④まで」の誤認識です。



08:30 「pull＝引っ張ってくる」

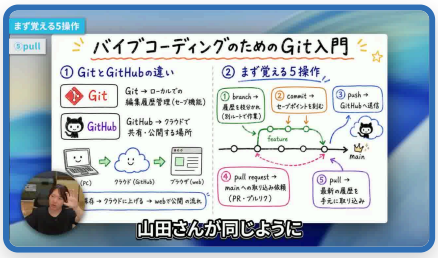
チーム開発でpullが必要になる理由

話者の例では、安野ブランチと山田ブランチがそれぞれpushしてプルリクを出し、両方がマージ（採用）されます。このとき自分の手元には自分の機能しかなく、山田機能は入っていません。手元をクラウド上の正しい歴史にそろえるにはpullが必要です。 [08:45–09:50]

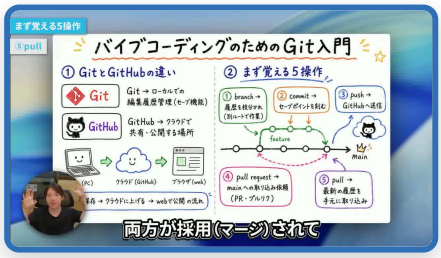
安野の手元（PC）	push・PR → / ← pull	クラウド上のmain（GitHub・正しい歴史）	← push・PR	山田さんの手元（別のノートPC）
安野機能：あり 山田機能：なし		安野機能：マージ済み 山田機能：マージ済み		山田機能：あり

話者の口頭の例を整理した図（画面にこの図はありません）。両方がマージされた後、安野の手元に山田機能を持ってくるにはpullが必要です。

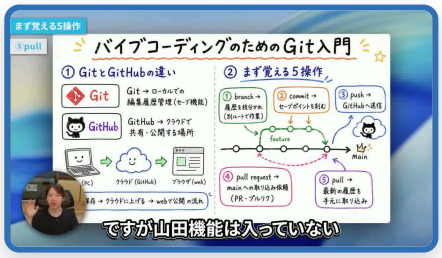
[08:45–09:50]



09:06 「山田さんが同じように」



09:14 「両方が採用（マージ）されて」



09:32 「ですが山田機能は入っていない」

PART 3 ／ 5操作の整理

クラウドとのやり取りは push と pull

クラウドとのやり取り

push　クラウドへ送る

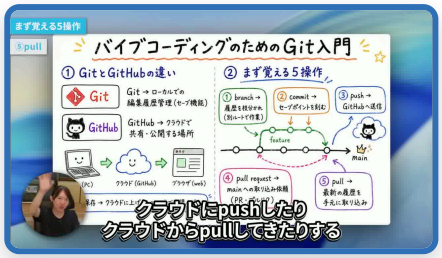
pull　クラウドから取ってくる

そのほかの操作

branch　ブランチを切る

commit　セーブポイントを打つ

pull request　PRを出す



09:52　「クラウドにpushしたり クラウドからpullしてきたりする」

話者は、push・pullの関係と、branch・commit・pull requestの一連の流れとして整理しています。 [09:50–10:06]

04

PART 4　10:06 – 11:31

注意点：conflict（衝突）は人間が決める

もう1つだけ重要な概念がconflict（衝突）。話者はホームページのテーマカラーを例に、口頭とテロップで説明します（専用の図はなし）。 [10:06–11:31]

安野ブランチ

テーマカラーを「赤」に

ブランチを切って赤くする作業をし、commit・push・プルリクを送る。

山田さん

テーマカラーを「青」に

その間に、青にする作業をしてpushする。 [10:06–11:31]

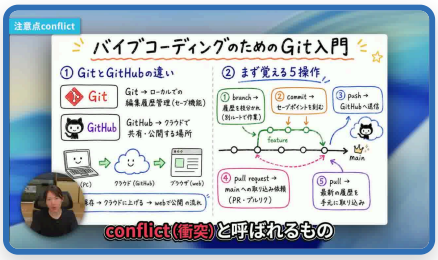
QUESTION

同じ項目が赤と青に分かれたら？

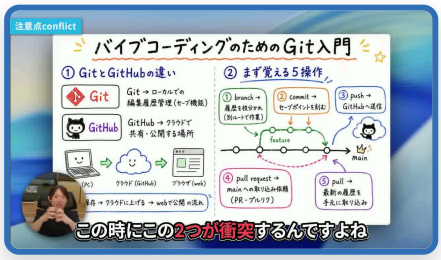
→

そのままでは取り込めない

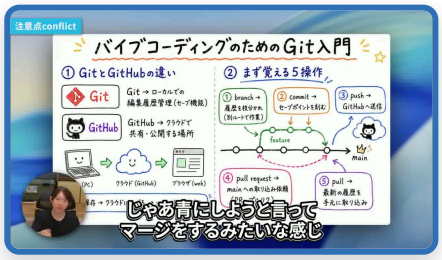
conflict状態になり、どうするか結論を出すようGitがユーザーに伝えてきます。人間が、たとえば青に決めてマージします。 [10:06–11:31]



10:11　「conflict（衝突）と呼ばれるもの」



10:49　「この時にこの2つが衝突する」



11:27　「じゃあ青にしようと言ってマージ」

05 PART 5　11:31 – 12:32
Claude Codeで、実際にやってみる

話者はbranch→commit→push→pull request→pullの一連の流れをやってみる、と予告して実演に移ります。 [11:31–11:49]

予告した流れ
5操作をひと通り

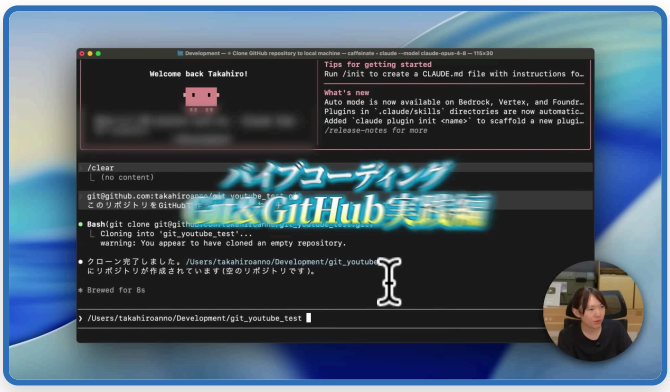
branch → commit → push → pull request → pull
（字幕は「bransh」と誤記）

この動画で画面に映った操作
クローンからcommit・履歴表示まで

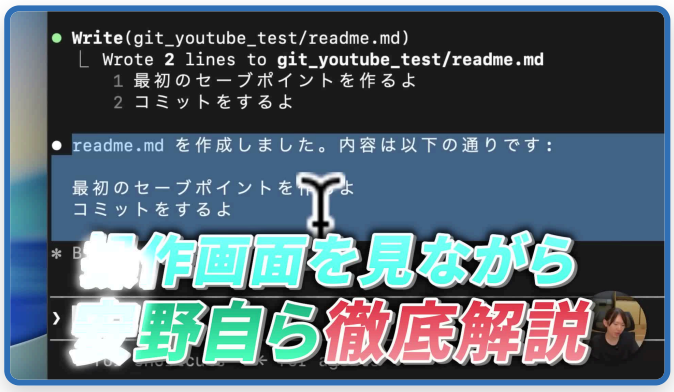
branch・push・pull request・pullの操作は映っていません。 [11:31–11:49]

STEP 1　空のリポジトリをクローンし、readme.mdを作る

ターミナル上のClaude Codeに、GitHub上の空のリポジトリ`git_youtube_test`のアドレスを渡してクローンを依頼。`git clone`が実行され、空のリポジトリだという警告が出ます。続けて「最初のセーブポイントを作るよ」「コミットをするよ」と書いたテキストファイルを頼み、readme.mdを作らせます。 [11:49–11:59]



11:49　Claude Codeでクローン完了。空のリポジトリの警告（テロップが重なる）



11:57　readme.mdに2行を書き込んだ結果

```
● Bash(git clone git@github.com:…/git_youtube_test.git)
  warning: You appear to have cloned an empty repository.
● Write(git_youtube_test/readme.md)  Wrote 2 lines
```

終わった状態：readme.mdが上記の2行で作成されている。

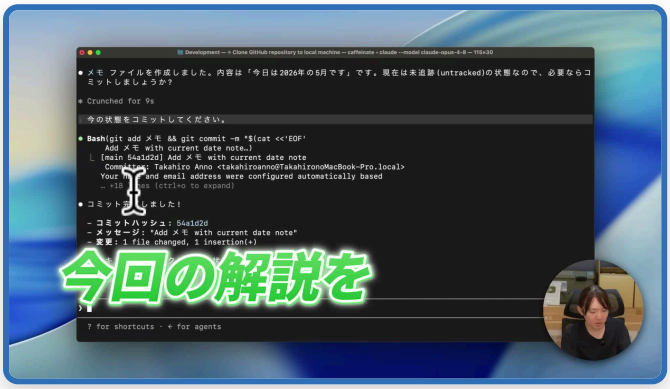
確認点：クローンの指示文はテロップで一部隠れて読めず、クローンからreadme作成までの間はカットされています。

PART 5 ／ 実演のつづき

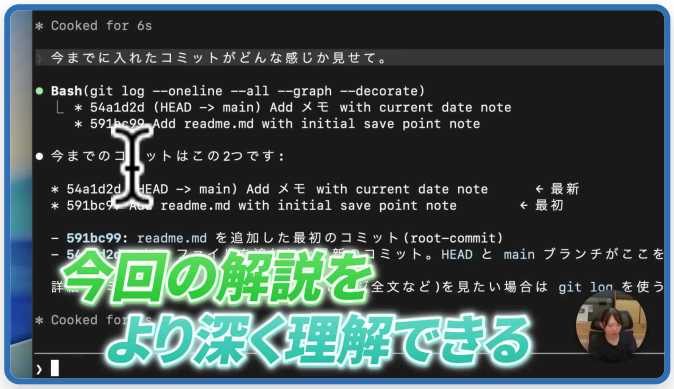
STEP 2　メモファイルをcommitし、履歴を見る

始めの状態：「今日は2026年の5月です」と書いたメモファイルが未追跡（untracked）で、Claude Codeが「必要ならコミットしましょうか？」と尋ねています。 [12:01–12:12]

「今の状態をコミットしてください。」と指示すると、`git add メモ && git commit`が実行されます。続けて「今までに入れたコミットがどんな感じか見せて。」と頼むと、`git log --oneline --all --graph --decorate`で履歴が表示されます。音声では、今いるブランチがmainであることも確認しています。 [12:01–12:12]



12:04　commit完了。[main 54a1d2d] Add メモ with current date note



12:08　git logで2つのコミットが並ぶ

コミット	メッセージ	位置づけ
54a1d2d	Add メモ with current date note	最新。HEAD -> main
591bc99	Add readme.md with initial save point note	最初のコミット（root-commit）

> **終わった状態：**mainに2つのコミットが並び、最新の54a1d2dにHEAD -> mainが付いている（変更は1 file changed, 1 insertion）。

確認点：readme.mdの最初のcommit（591bc99）、メモの作成、ブランチ確認の画面は映っていません。591bc99はgit logの出力で存在を確認できます。



12:12　次回予告のテロップ

Git&GitHub～実践編～ 近日公開!

最後にテロップで実践編を予告し、チャンネル登録と高評価を呼びかけて終わります。 [12:12–12:32]